Human Verification: `Containers.ProofSearchN`

N-ary Proof Search Trees as a Directed Container

Machine-generated from `lean/ProofSearchN.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.ProofSearchN` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **1**

2 Shapes: finitely-branching rose trees **2**
2.1 Inductive type: `RTree` . . . 2

3 Positions: paths in a rose tree **2**
3.1 Inductive type: `RPos` . . . 2

4 The directed-container operations **3**
4.1 Definition: `RTree.sub` . . . 3
4.2 Definition: `RTree.shift` . . . 3

5 Directed-container laws **4**
5.1 Theorem: `RTree.shift_root` (D3) . . . 4
5.2 Theorem: `RTree.sub_shift` (D4) . . . 4
5.3 Theorem: `RTree.shift_assoc` (D5) . . . 4

6 The directed container **5**
6.1 Definition: `proofSearchN` . . . 5

7 Translation conventions **5**

1 Overview

This file generalises `Containers.ProofSearch` from *binary* trees to **finitely-branching rose trees** (arbitrary arity at each node). The directed-container structure is identical in spirit: shapes are rose trees, positions are paths from root to a node, and the comonad operations come from path concatenation.

Encoding choice. Each node carries a branching factor $n : \mathbb{N}$ and a function assigning a child tree to each index $i : \mathrm{Fin}(n)$. The `Fin n` encoding avoids nested-inductive complications (which `List` would introduce) and lets structural recursion work directly on `RTree` and `RPos` without well-founded recursion or fuel arguments.

A **leaf** (dead end or success) is `node 0 f` where $f : \mathrm{Fin}(0) \to \mathrm{RTree}$ is vacuously defined. The only position in a leaf is `here`.

Relationship to `STree`. `STree` embeds into `RTree`: a leaf maps to `node 0 !` and `node` $\ell\, r$ maps to `node 2 ! [`ℓ'`,` r'`]`. This is not a strict sub-case since `STree` forces exactly 0 or 2 children.

2 Shapes: finitely-branching rose trees

2.1 Inductive type: `RTree`

Definition 2.1 (Rose tree)**.** A **finitely-branching rose tree** is a node with a branching factor $n : \mathbb{N}$ and a family of n child trees:

$$\mathrm{RTree} \;::=\; \texttt{node}(n : \mathbb{N},\ \mathrm{Fin}(n) \to \mathrm{RTree}).$$

A leaf (dead end) is $\texttt{node}(0, f)$ where f is vacuous.

Lean.

```
inductive RTree where
  | node : (n : Nat) → (Fin n → RTree) → RTree
```

Haskell.

```
data RTree = Node Int [RTree]    -- n children stored as a list
  deriving (Show, Eq)

-- A leaf is Node 0 [].
-- In Lean, children are indexed by Fin n (a bounded integer type).
-- Haskell approximates this with a plain list.
```

Fidelity note. In Lean, the children are given by a function $\mathrm{Fin}(n) \to \mathrm{RTree}$ where n is stored in the tree. This allows structural recursion on `RTree` and `RPos` directly. In Haskell, a list of children is the natural representation but cannot enforce the branching-factor invariant at the type level.

3 Positions: paths in a rose tree

3.1 Inductive type: `RPos`

Definition 3.1 (Position in a rose tree)**.** A **position** in an `RTree` is a path from the root to a node:

- `here`: the root (valid for any tree);
- `child`(i, p): descend into child i (where $i : \mathrm{Fin}(n)$) and continue along path p in that child.

For a leaf (`node 0 f`), there are no valid $i : \mathrm{Fin}(0)$, so the only position is `here`.

Lean.

```
inductive RPos : RTree → Type where
  | here  : RPos t
  | child : (i : Fin n) → RPos (children i) → RPos (.node n children
      )
```

Haskell.

```haskell
data RPos t where
  Here  :: RPos t
  Child :: Int → RPos child → RPos (Node n children)
-- Approximation (GADTs omitted for readability):
data RPos = Here | GoChild Int RPos
```

Fidelity note. In Lean, `RPos` is indexed by the tree. The `child` constructor is only well-typed when the tree is a `node n children`, and i must be a valid index in `Fin n`. Haskell's approximation loses both the tree-index and the bound on i.

4 The directed-container operations

4.1 Definition: `RTree.sub`

Definition 4.1 (Sub-tree at a position)**.** The sub-tree at position p in tree t:

$$t \downarrow \texttt{here} = t,$$
$$(\texttt{node}\, n\, \text{children}) \downarrow \texttt{child}(i, p) = \text{children}(i) \downarrow p.$$

Lean.

```lean
def RTree.sub : (t : RTree) → RPos t → RTree
  | t, .here ⇒ t
  | .node _ children, .child i p ⇒ (children i).sub p
```

Haskell.

```haskell
sub :: RTree → RPos → RTree
sub t              Here          = t
sub (Node _ cs)  (GoChild i p) = sub (cs !! i) p
sub (Node 0 _)   (GoChild _ _) = error "leaf␣has␣no␣children"
```

4.2 Definition: `RTree.shift`

Definition 4.2 (Shift (path concatenation))**.** The shift $p \oplus q$ embeds position q in the sub-tree at p back into the whole tree:

$$t.\texttt{shift}(\texttt{here}, q) = q,$$
$$(\texttt{node}\, n\, \text{children}).\texttt{shift}(\texttt{child}(i, p), q) = \texttt{child}(i,\ \text{children}(i).\texttt{shift}(p, q)).$$

Lean.

```lean
def RTree.shift : (t : RTree) → (p : RPos t) → RPos (t.sub p) →
    RPos t
  | _, .here, q ⇒ q
  | .node _ children, .child i p, q ⇒ .child i ((children i).shift p
      q)
```

Haskell.

```haskell
shift :: RTree → RPos → RPos → RPos
shift _             Here          q = q
shift (Node _ cs) (GoChild i p) q = GoChild i (shift (cs !! i) p q)
```

5 Directed-container laws

5.1 Theorem: `RTree.shift_root` (D3)

Theorem 5.1 (D3: right identity)**.** Shifting by the root of the sub-tree recovers the original position:

$$t.\mathtt{shift}(p, \mathtt{here}) \;=\; p \quad \text{for all } p : \mathrm{RPos}(t).$$

Lean.

```
def RTree.shift_root : (t : RTree) → (p : RPos t) →
    t.shift p .here = p
```

Haskell.

```
-- D3: prop_shift_root t p = shift t p Here == p
```

5.2 Theorem: `RTree.sub_shift` (D4)

Theorem 5.2 (D4: sub-trees nest along the shift)**.**

$$t \downarrow (p \oplus q) \;=\; (t \downarrow p) \downarrow q \quad \text{for all } p : \mathrm{RPos}(t),\ q : \mathrm{RPos}(t \downarrow p).$$

Lean.

```
def RTree.sub_shift : (t : RTree) → (p : RPos t) → (q : RPos (t.sub
   p)) →
    t.sub (t.shift p q) = (t.sub p).sub q
```

Haskell.

```
-- D4: prop_sub_shift t p q = sub t (shift t p q) == sub (sub t p) q
```

5.3 Theorem: `RTree.shift_assoc` (D5)

Theorem 5.3 (D5: shift is associative)**.** Path concatenation is associative (with transport along D4):

$$t.\mathtt{shift}(p \oplus q,\ D4^{-1} \triangleright r) \;=\; t.\mathtt{shift}(p,\ (t \downarrow p).\mathtt{shift}(q, r))$$

for all $p : \mathrm{RPos}(t)$, $q : \mathrm{RPos}(t \downarrow p)$, $r : \mathrm{RPos}((t \downarrow p) \downarrow q)$.

Lean.

```
def RTree.shift_assoc : (t : RTree) → (p : RPos t) → (q : RPos (t.
   sub p)) →
    (r : RPos ((t.sub p).sub q)) →
    t.shift (t.shift p q) ((t.sub_shift p q).symm [> r) =
      t.shift p ((t.sub p).shift q r)
```

Haskell.

```
-- D5 (transport suppressed):
-- prop_shift_assoc t p q r =
--   shift t (shift t p q) r == shift t p (shift (sub t p) q r)
```

Fidelity note. As in `ProofSearch`, the Lean statement requires r to be transported along sub_shift (D4) before the left-hand shift can consume it. In the rose-tree case, the `child` constructor recurses into the appropriate child and the recursion is structurally simpler than for `STree` (one constructor instead of `left`/`right`).

6 The directed container

6.1 Definition: `proofSearchN`

Definition 6.1 (The `proofSearchN` directed container)**.** Define the directed container of N-ary proof-search trees as:

$$\text{proofSearchN} \;=\; (\text{RTree},\ \text{RPos},\ \texttt{here},\ \texttt{RTree.sub},\ \texttt{RTree.shift})$$

with:

- D1: $t \downarrow \texttt{here} = t$ (by rfl);
- D2: $t.\texttt{shift}(\texttt{here}, q) = q$ (by rfl);
- D3: `RTree.shift_root`;
- D4: `RTree.sub_shift`;
- D5: `RTree.shift_assoc`.

Lean.

```
def proofSearchN : DirectedContainer where
  Shape := RTree
  Pos := RPos
  root := fun _ ⇒ .here
  sub := RTree.sub
  shift := RTree.shift
  d1 := fun _ ⇒ rfl
  d4 := RTree.sub_shift
  d2 := fun _ _ ⇒ rfl
  d3 := RTree.shift_root
  d5 := RTree.shift_assoc
```

Haskell.

```
proofSearchN :: DirectedContainer RTree RPos
proofSearchN = DirectedContainer
  { shape = RTree
  , pos   = RPos
  , root  = const Here
  , sub   = sub
  , shift = shift
  -- D1: sub t Here = t                (by definition)
  -- D2: shift t Here q = q            (by definition)
  -- D3: shift t p Here = p            (RTree.shift_root)
  -- D4: sub t (shift t p q) = sub (sub t p) q   (RTree.sub_shift)
  -- D5: shift t (shift t p q) r = shift t p (shift (sub t p) q r)
  }
```

7 Translation conventions

1. **Fin-indexed children.** Lean uses $\text{Fin}(n) \to \text{RTree}$ for children, enabling structural recursion without well-founded recursion or `List`. Haskell uses a plain list; the branching-factor bound is not enforced.
2. **Structural recursion.** Both `RTree.sub` and `RTree.shift` recurse on paths by structural induction, with `here` as the base case and `child i p` as the recursive case. This is simpler than the binary case (one recursive constructor vs. two).

3. **Relationship to binary trees.** `proofSearch` is a special case of `proofSearchN` with branching factors restricted to $\{0, 2\}$. The Lean formalisation keeps them separate to avoid the round-trip gap that would arise from an embedding.
4. **D1 and D2 definitional.** As in the binary case, the first case (`here`) in each recursive definition is definitionally equal to the expected value, so D1 and D2 are proved by rfl.
5. **Proof bodies omitted.** Only statements are translated.